Eyes Wide Shut: Developing Adaptive Enemy AI for Stealth Gameplay

GDEV60001 GAMES DEVELOPMENT PROJECT

Hennio Monteiro Dias da Silva
SUPERVISOR: DAVIN WARD
SECOND SUPERVISOR: LUKE BARSBY

Contents

Abstract

The stealth genre has long relied on Finite State Machines (FSM) to govern non-player character (NPC) behaviour. While reliable, FSMs suffer from deterministic predictability, allowing players to exploit static patterns rather than engaging with a truly intelligent adversary. This dissertation investigates whether Case-Based Reasoning (CBR) can be integrated into a real-time game environment to create an adaptive AI capable of learning from failure.

The research involved the development of a Unity-based artefact featuring a sensory system governed by the Inverse Square Law and a bespoke CBRManager. This system enabled an NPC to record instances where it lost the player (e.g., in darkness) and autonomously deploy counter-measures (e.g., interacting with light switches) upon subsequent encounters.

Testing was conducted through a mixed-methods approach, triangulating quantitative telemetry data with qualitative player feedback. The results demonstrated that the agent successfully executed "One-Shot Learning," adapting its strategy within 40 seconds of an initial failure. While the telemetry confirmed the algorithmic efficacy of the system, player surveys revealed a disconnect between functional competence and perceived intelligence, highlighting the necessity of audio-visual feedback to communicate intent.

The study concludes that CBR is a computationally efficient architecture for creating adaptive stealth gameplay, offering a viable alternative to resource-intensive Machine Learning models. The findings suggest that while logic is the foundation of intelligence, the performance of that intelligence is critical for player immersion.

Introduction

The stealth genre is fundamentally defined by the tension of avoiding detection while navigating an environment populated by hostile agents. Unlike action-oriented genres where AI primarily serves as a mechanical obstacle to be overcome through force, stealth AI acts as a systemic puzzle that requires observation and exploitation. Historically, titles such as *Thief: The Dark Project (1998)* and *Metal Gear Solid (1998)* established the foundational pillars of visual and auditory perception that still govern the genre today. These systems rely on raycasting for line-of-sight and radial distance checks for sound, creating a “cat and mouse” loop that defines the player experience.

However, a persistent challenge within modern stealth design remains the predictability of non-player character (NPC) behaviour. Standard AI implementation frequently utilises static patrol routes and binary detection states which, while functional, allow for ‘metagaming’. Metagaming is a process where players exploit rigid, scripted patterns rather than engaging with a believable, reactive opponent. When an agent’s behaviour is restricted to a simple Finite State Machine (FSM) without a memory of past player interactions, the systemic puzzle becomes trivial once the player identifies the loop.

This research project, titled “Eyes Wide Shut,” seeks to address this predictability by shifting the focus from scripted routines to a technical implementation of adaptive AI. Rather than merely challenging player “immersion”, a subjective metric, this project focuses on the objective performance and efficiency of an AI framework that dynamically modifies its behaviour based on player strategy. The core technical problem is: How can a module AI system be engineered to recognise and counteract repeated player tactics in real-time within the Unity engine?

By comparing a “Standard FSM” against an “Adaptive AI” framework, this dissertation will evaluate the technical complexity required to move beyond static patrolling. The proposed system will incorporate a “Heatmap” of the player’s last known locations, allowing agents to prioritise search parameters in areas where the player has previously been detected. To validate this, the project utilises comprehensive AI Debug Tools to visualise field of view (FOV) cones, suspicion radii, and decision-tree logic transitions. This technical approach ensure that the resulting artefact is not just a game prototype, but a scalable demonstration of intelligent gameplay programming suitable for industry-standard roles in AI development.

Aims and Objectives

The primary aim of this project is to design, implement, and evaluate a technical framework for Adaptive Enemy AI within a stealth gameplay context using Unity and C#. This research specifically targets the mitigation of “metagaming”, by replacing static, scripted patrol routes with a dynamic system that reacts to and “remembers” player behaviour to maintain a consistent technical challenge.

Objectives

To achieve this aim, the following objectives will be met:

- Develop a robust Sensory System: Creating an AI perception model using raycasting for Line-of-Sight (LoS) detection and radial triggers for auditory perception.
- Implement a Modular Finite State Machine (FSM): Engineer a clear state-logic framework in C# to manage transitions between Idle, Suspicious, and Alerted behaviours.
- Engineer the Adaptive “Memory” Logic: Develop a pattern-recognition system (e.g., a “Heatmap” or coordinate-based list) that stores the player’s last known locations and prioritises those areas during the search phases.
- Create a custom AI Debug and Visualisation Tools: Build an in-editor developer interface (using onDrawGizmos) to visualise FOV cones, suspicion growth, and navigation paths for objective performance analysis.

Literature Review

Section 1: The Evolution of Stealth AI Architecture

1.1 Introduction: The Role of the adversary in Stealth Systems

The core of any stealth game lies in the asymmetrical relationship between the player and the environment. Unlike action-oriented titles where non-player characters (NPCs) often serve as “cannon fodder” or immediate mechanical obstacles, stealth AI functions as a systemic puzzle that must be decoded through observation. Throughout history, stealth AI has evolved from rigid, binary systems into complex, hierarchical architectures that attempt to simulate human-like cognition, perception, and reaction. The history of this evolution is defined by a transition from scripted, top-down loops towards emergent, adaptive behaviours designed to maintain tension and mitigate the risk of “metagaming” (Millington, 2021), the process by which players exploit predictable AI limitations to bypass challenges.

1.2 Early Foundations: *Thief vs. Metal Gear Solid*

The late 1990s marked a pivotal era for stealth design, characterised by two distinct approaches to AI architecture. Hideo Kojimas's *Metal Gear Solid (1998)* utilised a top-down, arcade inspired approach where NPCs operated on highly structured, binary detection loops. In this model, enemy perception was largely limited to a 2D "view" displayed on the player's radar. If the player entered the cone, a binary state change occurred: from "Patrol" to "Alert". While effective for creating immediate pressure, these early LoS (Line of Sight) systems lacked nuance; enemies were often unable to perceive depth or varying degrees of visibility, leading to a "hide and seek" loop that felt more mechanical than organic.

In contrast, Looking Glass Studios' *Thief: The Dark Project (1998)* introduced a revolutionary "sensory" architecture via its Dark Engine. *Thief* was arguably the first PC title to integrate light and sound as a core game mechanic rather than cosmetic effects (Looking Glass Studios, 1998). Instead of a binary vision cone, *Thief* implemented a robust 3D sound system where AI agents used "A-Star" pathfinding to investigate the coordinates of specific sound events. By utilizing light-sensitive "Light Gems" and complex auditory radii, *Thief* moved away from the arcade style of *MGS* and toward an "Immersive Sim" model, where NPCs serve as a foil to the player's cunning. This transition laid the groundwork for modern sensory AI, establishing that a believable opponent must react to multiple environmental stimuli, and not just visual contact.

1.3 Finite State Machines (FSM): The Standard Logic Backbone

The most prevalent architecture in stealth AI remains the Finite State Machine (FSM). An FSM organises an agent's behaviour into a limited number of discrete states (e.g., Idle, Patrol, Suspicious, Alert, Combat), where the agent can only exist in one state at any given time. Transitions between these states are triggered by specific environmental inputs or Boolean conditions, such as the following:

```
If (playerSeen == true) currentState = State.Alert;

If (unfamiliarNoiseDetected == true) currentState = State.Suspicious;
```

In stealth gameplay, the FSM is essential for defining the "Search Loop". Millington (2021) describes this as the core decision-making logic that allows an AI to transition from a routine patrol to an investigation phase when a sensory threshold is met (Millington, 2021). However, FSMs suffer from two significant technical limitations: predictability and state explosion.

- Predictability: Because FSMs are rule-based, players can easily learn the exact transition triggers, leading to "metagaming" where the AI becomes a predictable clockwork machine rather than a reactive threat.
- State Explosion: As developers add more complex behaviours (e.g., Radio for Backup, Investigate Last Known Position, Check Locker), the number of required transitions grows geometrically. A system with 10 states might require hundreds of explicit arrows to manage every possible transition, making the code brittle and difficult to debug (Yannakakis and Togelius, 2018).

1.4 Modern Advancements: Behaviour Trees and Utility AI

To solve the "state explosion" issues of FSMs, modern AAA titles like *Halo 2* and *Hitman (2016-2021)* shifted toward Behaviour Trees (BT) and Utility AI. Unlike the rigid connections of an FSM, a Behaviour Tree is a hierarchical structure that "ticks" from a root node down through various branches.

- Modularity: BTs allow for “Selector” and “Sequence” nodes that naturally handle interrupts. For example, a high-priority “Find Cover” branch can automatically override a “Patrol” branch without requiring a spaghetti-mess of manual transitions (Champandard, 2007).
- Emergent Behaviour: Modern stealth titles, particularly the *Hitman* series, often combine BTs with “Utility” scoring systems. In a Utility AI model, every possible action is assigned a mathematical “weight” based on environmental variables (e.g., proximity to player, ammo count, health). The AI then performs the action with the highest score, creating behaviours that appear significantly more “human” and less scripted than traditional FSM-driven guards (Isla, 2005).

1.5 Technical Implementation Analysis: FSM vs. Behaviour Tree Logic

To understand the necessity of adaptive AI, one must first analyse the underlying code structures that govern standard NPC behaviour. In a development environment like Unity using C#, the choice between a Finite State Machine (FSM) and a Behaviour Tree (BT) is not merely an architectural preference but a fundamental decision that dictates the scalability and “metagaming” potential of the game.

1.5.1 Deconstructing the FSM in C#

The standard FSM implementation typically utilises an *enum* to define the state space and a *switch* statement within the *Update()* loop to process logic. Below is a conceptual deconstruction of a standard stealth guard’s logic:

```
public enum GuardState { Idle, Patrol, Suspicious, Alert }
public GuardState currentState = GuardState.Idle;

void Update() {
    switch (currentState) {
        case GuardState.Idle:
            if (CanSeePlayer()) currentState = GuardState.Alert;
            else if (HearSound()) currentState = GuardState.Suspicious;
            break;
        case GuardState.Patrol:
            MoveToWaypoints();
            if (CanSeePlayer()) currentState = GuardState.Alert;
            break;
        case GuardState.Suspicious:
            InvestigateLastSound();
            if (TimerDone) currentState = GuardState.Patrol;
            break;
        case GuardState.Alert:
            ChasePlayer();
            if (!CanSeePlayer()) currentState = GuardState.Suspicious;
            break;
    }
}
```

While this structure is computationally efficient and easy to debug, its technical fragility lies in its linear reactivity (Champandard, 2007). Each state is a “silo”; the guard in the *Patrol* state has no inherent awareness of the *Suspicious* state’s variables. If a developer wishes to add a “Radio for Help” behaviour that occurs only if the guard is in the *Alert* state and near a terminal, they must manually wire a new transition and check for those specific Booleans within the existing switch case. This leads to the “State Explosion” problem, where the complexity of managing transition becomes a barrier to creating believable, non-scripted AI.

1.5.2 Behaviour Trees and Functional Modularity

Conversely, a Behaviour Tree offers a hierarchical, tick-based approach. Instead of a state-to-state jump, the AI "ticks" from the root and evaluates a sequence of conditions every frame. For a stealth agent, this allows for prioritised interruption.

In a Behaviour Tree, a "Selector" node can prioritise a "Safety" branch (e.g., Check Health or Find Cover) over a "Patrol" branch. The AI does not "switch states" in a tradition sense, but rather the "Safety" branch simply returns a "Success" status, preventing the "Patrol" branch from ever being reached during that tick. This modularity is what allows modern stealth titles to feel more organic. If the player creates a distraction, the Behaviour Tree can simply insert a "Distraction Response" task into its current sequence without needing to rewrite the entire patrol logic.

1.5.3 Performance and Scalability: The Developer's Dilemma

From a performance perspective, FSMs are virtually "free" in terms of CPU overhead (Millington, 2021). In a simulation with dozens of guards, an FSM-based approach ensures that the game maintains a high frame rate on lower-end hardware. However, the cost is transferred to the developer's time and the player's experience. The rigidity of the FSM is exactly what facilitates "metagaming". Because the transitions are hard-coded, a plyer can learn that "if I throw a rock, the guard will stand in Position X for exactly 5 seconds before returning to a Patrol State".

The adaptive AI proposed in this dissertation seeks a middle ground: using a modular FSM for its performance benefits while introducing a Data-Driven Memory Layer. This layer allows the FSM to "remember" previous player locations, effectively simulating the reactivity of a Behaviour Tree without the architectural overhead.

1.6 Case Study: Splinter Cell: Blacklist and Last Known Position (LKP)

A landmark achievement in mitigating metagaming through adaptive AI is the "Last Known Position" (LKP) system, most notably refined in Splinter Cell: Conviction and Blacklist. In traditional FSM-based stealth, once a player breaks line of sight, the AI typically returns to a "Search" state at the exact point of disappearance or resets to a "Patrol" route after timed delay. This predictability allows players to use "ghosting" tactics-merely breaking sight for a few seconds to "reset" the AI's aggression.

The LKP system introduces a ghost silhouette, which is a representation of data about where the AI thinks the player is. Technically, this is implemented by catching the *Vector3* coordinates of the player at the exact frame the AI's detection value drops to zero. Instead of resetting, the AI initiates a "Flank and Flush" behaviour. Agents are programmed to not just move to the LKP, but to use pathfinding to navigate the vantage points surrounding that coordinate.

This creates a "memory-driven" adaptation that forces the player to move constantly (Ubisoft Toronto, 2013). In Blacklist, the AI's effectiveness is amplified by "Group Knowledge". If one guard identifies an LKP, that coordinate is broadcast to the "Blackboard" of all nearby agents. This prevents the "conga line of death" exploit in common games like Skyrim, where players can kill enemies one by one in the same doorway because the AI lacks a shared memory of the danger zone (Millington, 2021). For the "Eyes Wide Shut" project, adopting an LKP-style memory layer is essential for transforming the guard from a stationary observer into an active hunter that closes off the player's previous escape routes.

1.7 Deconstructing the A* Algorithm: Auditory Pathfinding

To facilitate the "investigation" phase of the adaptive, a robust navigation algorithm is required. The A* (A-Star) algorithm remains the industry standard for this task due to its efficiency in finding the shortest path between a guard and a suspicious sound source. A* functions by calculating a "Cost" for every potential "node" or tile (Dijkstra, 1959) on the navigation mesh (NavMesh).

This formula is expressed as:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$: The actual cost from the starting node (the guard's current position) to the current node n.
- $h(n)$: The "Heuristic" – an estimate of the cost from node n to the node goal (the sound source).

In the context of "Eyes Wide Shut," the A* algorithm is triggered the moment an auditory stimulus exceeds the AI's "Hearing Threshold". The guard does not "teleport" to the sound, but the engine generates a path of nodes that avoids obstacles. However, a standard A* Path is often too "perfect," leading to robotic movements that players can easily predict (Yannakakis and Togelius, 2018).

To make the AI more "human" and adaptive, this project will implement Path Smoothing and Node Jitter. Instead of taking the mathematically perfect centre of the path, the AI will add a slight random offset to its search coordinates. This ensures that if a player makes a noise in hallway twice, the guard will not talk the exact same pixel-perfect line both times, thereby increasing the difficulty of predicting the AI's movement and mitigating the metagaming loop.

Section 2: Sensory Perception Models

Section 2.1: The Geometry of Visual Perception: Field of View and Raycasting

In stealth gameplay, the most critical sensory input is visual perception. To simulate an agent's "eyes," developers must implement a system that balances mathematical accuracy with computational performance (Millington, 2021). In Unity, this is typically achieved through a combination of Trigonometric Field of View (FOV) checks and Raycasting for occlusion.

Section 2.1.1: The Dot Product and Angular Visibility

The first layer of vision is determining if the player's *Transform.position* falls within the agent's viewing frustum. This is not a simple distance check, as it requires calculating the angle between the agent's forward vector and the vector pointing towards the player.

Mathematically, this uses the Dot Product to find the cosine of the angle between two normalized vectors:

$$dot = A \cdot B = |A||B|\cos(\theta)$$

In C#, the implementation determines if the player is within a specific angle (e.g, 90°). If the dot product of the agent's forward vector and the direction to player vector is greater than the cosine of half of the FOV angle, the player is considered "potentially visible."

Section 2.1.2: Raycasting for Occlusion and Stealth Layers

Passing the angular check is insufficient for a stealth game, as the AI must also check for environmental obstacles. This is where Raycasting is employed. A "Ray" is cast from the agent's eye

coordinates to the player's pivot point. If the ray hits a collider with a "Wall" or "Cover" tag before reaching the player, the vision is blocked.

To increase technical complexity, this project will implement Multi-Point Raycasting. Instead of a single ray, to the player's centre, rays are cast to the head, torso, and feet. This allows for "partial visibility" (e.g, a guard seeing a player's feet under a door), which directly informs the suspicion meter.

Section 2.2: Auditory Perception: Simulating Sound Propagation

While vision is directional, sound is radial and omnidirectional. In the Dark Engine used for *Thief*, sound was simulated as "events" that travelled through a portal system. For "Eyes Wide Shut", we will utilise a Radial Event System within Unity.

Section 2.2.1: Noise Events and Attenuation Curves

Every player action whether its running, jumping, throwing an object generates a *NoiseEvent* with a *radius* and *intensity.* When an AI agent falls within this radius, it calculates the "perceived volume" based on a Linear Attenuation Curve:

$$V_p = V_s \times (1 - \frac{d}{r})$$

Where V_p is perceived volume, V_s is source volume, d is distance, and r is the maximum radius.

Section 2.2.2: Auditory Occlusion and Material Density

To prevent AI from hearing through three-foot concrete walls, a common "metagaming" exploit, the project implements Auditory Occlusion. Using a *Physics.Linecast*, the system checks for obstacles between the noise source and the guard. If an obstacle is detected, the noise intensity is reduced by a "Density Multiplier" associated with the wall's material. This forces the AI to investigate the direction of the sound rather than the exact source coordinate, creating a more believable search pattern.

Section 2.3: The Psychology of the Suspicion Meter: Balancing Fairness and Tension

The "Suspicion Meter" (or Detection Meter) acts as the bridge between raw mathematical data and player feedback. Historically, stealth games moved away from "Instant Detection" because it felt unfair and unpredictable to the player.

Section 2.3.1: Linear vs. Exponential Growth

The rate at which a meter fills is determined by variables such as distance, light level, and player stance. To maximise tension, "Eyes Wide Shut" uses exponential growth for detection. When a player is barely visible at long range, the meter fills up slowly; as they move closer, the rate increases exponentially. This gives the player a "window of opportunity" to dive into cover, a mechanic known as Coyote Time for stealth.

Section 2.3.2: The "Grace Period" and Social Stealth

As noted in the literature regarding *Hitman (2016)*, AI agents often have a "soft state" where they stare at the player before transitioning into a full suspicious state. This is technically a "Pre-Detection Timer." In my adaptive FSM, this timer will be dynamic. If the player has been caught in the same area multiple times, the "Grace Period" will shrink, effectively simulating an AI that is "on edge" and adapting to player aggression.

Section 2.4: Comparison of 2D vs. 3D Detection: Spatial Complexity and Performance

The transition from two-dimensional stealth environments to fully realised 3D spaces fundamentally altered the mathematical requirements for AI perception. In 2D stealth titles, such as *Mark of the Ninka* or early *Metal Gear* titles on the NES, detection is often calculated on a single horizontal plane. The "vision cone" is a simply 2D triangle defined by an origin point, a direction vector, and an angular width. Mathematically, determining if a player is detected in 2D involves a basic distance check followed by a 2D dot product calculation to verify the angle.

However, in a 3D environment like the mock CIA facility proposed for "Eyes Wide Shut," the AI must account for verticality, which introduces a third axis (Z) and significantly increases the complexity of occlusion checks (Yannakakis and Togelius, 2018). In 3D, the vision cone becomes a Frustum, a truncated pyramid that represents the field of view of a camera or an eye. To check if a player is within this 3D volume, the engine must perform a frustum-culling check, which is computationally more expensive than a 2D angular check.

The most significant technical challenge in 3D perception is depth and perspective. In 2D, "cover" is often binary: you are either behind a crate, or you are not. In 3D, the player can be partially obscured by a railing, hidden above a guard on a walkway, or prone in high grass. To handle this, 3D stealth AI uses Multi-Raycast Sampling. While a 2D guard might only need one "Line of Sight" (LoS) check, a 3D guard must cast rays to multiple "bones" or "sockets" on the player's character model. This project will implement a sampling system where rays are fired at the head, spine and left foot and right foot positions.

The technical trade-off here is performance. Firing four or five rays per guard every frame across twenty guards can lead to a significant CPU bottleneck. To optimise this, "Eyes Wide Shut" will implement a Sensory Tick-Rate. Instead of checking the vision every frame (60 fps), the AI will "sample" the world at a lower frequency (e.g., 10 times per second), distributing the raycast load over multiple frames to maintain a high render rate while preserving the illusion of constant vigilance.

Section 2.5: Lighting, Global Illumination, and Visibility Sampling

In a stealth game, light is not merely a visual asset, but it is a primary variable in the detection equation. The Dark Engine used in *Thief (1998)* pioneered the "Light Gem," which sampled the light level at the player's exact coordinate to determine their visibility. In modern engines like Unity, we use Global Illumination and Light Probes to achieve a similar, though more technically sophisticated, effect.

The adaptive AI in this project calculates a *visibilityMultiplier* based on the ambient light surrounding the player. Technically, this is achieved by sampling the nearest Light Probe data at the player's position. Unity stores light information in spherical harmonics, which can be queried in C# to return a brightness value between 0.0 (total darkness) and 1.0 (full illumination).

The detection formula for “Eyes Wide Shut” is structured as:

$$D_{rate} = B \times (1 - \frac{d}{d_{max}}) \times L_{factor}$$

Where:

- D_{rate} is the speed at which the suspicion meter fills.
- B is the base detection speed of the guard.
- d is the distance to the player.
- L_{factor} is the light level sampled from the environment.

This logic creates an adaptive challenge: if a player stays in the shadows, L_{factor} remains low, causing the detection meter to fill at a negligible rate. However, if the player enters a lit hallway, the L_{factor} spikes, forcing the AI to transition to a suspicious state almost instantly, this reinforces the “systemic puzzle” aspect of stealth, where the player must treat light sources as physical hazards.

Furthermore, the “adaptive” nature of my project expands this by allowing guards to interact with the light. If an AI agent enters a suspicious state after hearing a noise in a dark room, its first “Search Behaviour” will be to navigate to a light switch or use a flashlight. This changes the L_{factor} of the room dynamically, removing the player’s environmental advantage and forcing a change in strategy. This moves the AI away from being a “static observer” and into a “proactive agent” that manipulates the game world to find the intruder.

Section 2.6: Sensory Overload and Distraction Priorities: Conflict Resolution Logic

In a complex stealth environment, an AI agent is rarely processing a single stimulus in isolation. A common failure in lower-tier stealth AI, and a primary target for “metagaming,” is the “Sensory Stutter.” This occurs when an agent receives two competing signals of equal weight (e.g., seeing the player’s shadow while simultaneously hearing a distraction to the left) and oscillates rapidly between states, breaking the player’s immersion and the AI’s technical credibility (Champandard, 2007). To resolve this, the “Eyes Wide Shut” framework implements a Hierarchical Stimulus Priority System within its modular FSM.

The technical problem lies in “Stimulus Ranking.” In my implementation, every sensory event is assigned a Priority Integer, and a Decay Timer. Visual stimuli are hard-coded with the highest priority (Priority 1), as they represent confirmed threat. Auditory stimuli (Priority 2) and environmental changes, such as an open door that was previously closed (Priority 3), follow in descending order.

When the AI’s “Blackboard” receives a new stimulus, it performs a Comparison Check:

```
void OnSensoryEvent(Stimulus newInbound) {
    if (newInbound.priority < currentFocus.priority) {
        // Interrupt current behavior for the higher priority event
        ChangeState(State.Alert);
    } else if (newInbound.priority == currentFocus.priority) {
        // Handle equal weights: Focus on the closest proximity
        if (newInbound.distance < currentFocus.distance) {
            UpdateFocus(newInbound);
        }
    }
}
```

This logic prevents the AI from being easily “led away” by a simple rock throw if it has already caught

a glimpse of the player. In industry-standard AI, such as the system described by Damian Isla for *Halo 2*, this is known as Action Selection. The AI must maintain a “Commitment Timer” to a specific stimulus (Isla, 2005) to prevent it from looking like a flickering machine.

Furthermore, the “adaptive” component of my dissertation introduces Distraction Fatigue. If a player repeatedly uses the same distraction (e.g., throwing a bottle into the same corner three times), the AI’s *AuditorySensitivity* variable for that specific coordinate begins to decay. Technically, this is achieved by storing the coordinates of the last three sound events in a *CircularBuffer<Vector3>*. If a new sound occurs within a 2-meter radius of a stored point, the AI’s *SuspicionGrowthRate* is halved. This forces the player to innovate rather than exploit the AI’s “curiosity” loop, effectively closing a major metagaming loophole and stimulating a guard who is “learning” that the noise is likely a ruse.

Finally, this section addresses Sensory Making. In a realistic technical model, loud ambient noises (like a passing truck or a heavy rain loop) should raise the noise “Noise Floor” of the AI. My project implements a *GlobalAcousticLevel* variable. When the environment is loud, the AI’s *HearingRadius* is dynamically shrunk:

$$R_{effective} = R_{base} \times (1 - L_{ambient})$$

This adds a layer of “Tactical Stealth” where the player must time their movements to coincide with environmental noise, further diversifying the gameplay loop away from static, predictable patterns.

Section 3: Adaptive AI and Pattern Recognition

Section 3.1: The problem of “Metagaming” and Exploitative Player Behaviour

In the context of game design, "metagaming" refers to the use of out-of-game knowledge or the exploitation of a game’s underlying mechanical weaknesses to gain an advantage. Within the stealth genre, metagaming manifests as a player’s ability to "solve" an AI agent not as a sentient threat, but as a predictable algorithm (Millington, 2021). When an NPC’s behaviour is governed by a static Finite State Machine (FSM) with no memory of past events, the player quickly identifies the "reset points" of the loop. For example, if a guard investigates a noise for exactly ten seconds before returning to a fixed patrol route, the player no longer treats that guard with caution; they treat the guard as a timer to be waited out.

The technical root of this problem is Deterministic Logic. Most standard stealth AI is deterministic given the same input (a sound at coordinates X, Y), it will produce the exact same output every time. This lack of variance is what facilitates exploitation. Players engage in "Trial and Error" gameplay where they intentionally trigger AI states to map out the boundaries of the NPC’s perception. As noted by Yannakakis and Togelius (2018), when the player can predict the AI's next move with 100% accuracy, the "suspense" required for a stealth experience collapse (Yannakakis and Togelius, 2018), and the game becomes a rote mechanical exercise rather than an immersive simulation.

Furthermore, the "Leash" effect in stealth AI contributes heavily to metagaming. Most NPCs are "leashed" to a specific zone or path to prevent them from wandering too far and breaking the level's flow. Players exploit this by standing just outside the AI's "Zone of Interest." An agent might chase a player to the edge of a doorway and then immediately turn around because its script forbids it from exiting the room. This "mechanical wall" shatters the illusion of a reactive adversary.

To counter this, "Eyes Wide Shut" proposes a shift toward Heuristic-Driven Reactivity. Instead of purely deterministic outcomes, the adaptive AI will incorporate "weighted randomness" and "state persistence." If a player is spotted, the AI should not merely "reset" after a timer; it should enter a permanent state of heightened awareness, modifying its detection speed or patrol frequency for the remainder of the session. By introducing these non-linear elements, the project aims to move the player away from "system exploitation" and back toward "tactical adaptation," where the player must respect the AI as a learning entity.

Section 3.2: Dynamic Patrol Adjustment: Heatmaps and NavMesh Modification

One of the most effective technical methods for breaking player-perceived patterns is the implementation of Dynamic Patrol Adjustment. In a standard stealth system, waypoints are baked into the scene at design time. In an adaptive system, these waypoints must be generated or modified at runtime based on player interaction data. The primary tool for this in "Eyes Wide Shut" is the Influence Map (or Heatmap).

An Influence Map is a grid-based data structure overlaid on the game world that stores values representing "Player Presence" (Millington, 2021). Every time the player is detected, or even just suspected, at a specific coordinate, a "Heat Value" is added to that cell in the grid. Over time, these values decay, but the cumulative data provides the AI with a visual representation of the player's preferred routes and hiding spots.

From a technical implementation standpoint in Unity, this involves the AI agents querying the Influence Map during their Patrol state. Instead of moving from Waypoint A to Waypoint B, the agent performs a Weighted Selection:

- The agent identifies all available waypoints within a specific radius.
- It cross-references these points with the Influence Map.
- Waypoints with a high "Heat Value" are given a higher priority in the pathfinding queue.

This results in an AI that naturally "gravitates" toward the areas where the player has been most active. This is not "cheating" by giving the AI the player's current location; rather, it is simulating Strategic Intuition (Champandard, 2007). If a guard finds a door open or hears a floorboard creak in the library twice, it is logically consistent for that guard to patrol the library more frequently. By modifying the *NavMeshAgent.destination* based on this heatmap data, the AI breaks the "static route" meta, forcing the player to constantly find new paths through the environment and maintaining the technical challenge throughout the play session.

Section 3.3: Simulated Learning: Case-Based Reasoning (CBR) and Pattern Recognition

To move beyond simple "Heatmaps," the AI must exhibit a form of simulated learning that feels coherent to the player. While true Machine Learning (ML) is often computationally prohibitive for real-time game environments due to the overhead of neural network inference, Case-Based Reasoning (CBR) provides a viable, data-driven alternative. CBR is a paradigm of artificial intelligence that solves new problems by remembering previous similar situations and adapting their solutions (Aamodt and Plaza, 1994). This is technically implemented as a "Memory Buffer" that records "Failure Cases" for the AI, specific instances where the player successfully bypassed a guard or escaped an alert.

When an AI agent "loses" the player, it initiates a Post-Mortem Data Event. This event generates a Struct containing the player's exit vector, the light level at the time of disappearance, and the specific state the AI was in. In future patrol cycles, the AI queries this "Library of Failures." If the current environmental conditions match a stored case (e.g., "Player escaped via the north vent in low light"), the AI's Inference Engine triggers a "Counter-Behaviour."

For example, the guard may spend more time looking *up* at the vents or proactively turning on a light in that sector. Technically, this is handled through a Similarity Function that compares the current state S to stored cases C:

$$Sim(S, C) = \sum_{i=1}^{n} w_i \times f_i(S_i, C_i)$$

Where w_i is the weight of a specific variable (like distance) and f_i is the scoring function. If the similarity score exceeds a certain threshold, the AI swaps its default patrol logic for a "High-Alert Search" logic. This effectively counters the player's ability to "metagaming" the environment, as the AI actively closes the loopholes the player relies on.

Section 3.4 Environmental Interaction and Smart Objects

Predictability in stealth games often stems from the environment being a "static stage" where only the player is the active agent. To elevate the technical complexity of a stealth game, the AI must treat the environment as a dynamic set of variables that can be manipulated to flush out the player. This involves the implementation of Smart Objects and an Environmental Blackboard.

Standard AI treats a closed door as a pathfinding obstacle. An adaptive AI treats it as a Signal (Millington, 2021). If a guard's *InitialState* recorded a door as "Closed," and it is now "Open," the AI's suspicion does not just tick up; it triggers a *ContextualInvestigate* command. This uses the *A* Algorithm* not just to find the player, but to "trace back" the likely path the player took to reach that door. By analysing the NavMesh nodes between the guard's last check and the current environmental change, the AI can project a "Search Cone" that is far more accurate than a random wander.

Furthermore, the AI can engage in Proactive Environment Manipulation. If the "Heatmap" discussed in Section 3.2 shows high player activity in a dark corridor, the AI's logic will prioritise "Securing the Zone." This might involve the AI locking a door, moving a physics object to block a known flanking route, or calling a second agent to perform a "Pincer Patrol". Technically, this is managed by the Global Blackboard which issues "Directives" to individual FSMs based on aggregate data. By allowing the AI to "edit" the level layout at runtime, the project ensures that the player can never truly "solve" the map, as the map itself evolves to counter their specific playstyle.

Section 3.5: Technical Synthesis: Closing the Metagaming Loop

The ultimate goal of combining CBR, Heatmaps, and Environmental Interaction is to create a feedback loop that mirrors human intuition. In academic literature, this is often referred to as the OODA Loop (Observe, Orient, Decide, Act) (Boyd, 1996). In "Eyes Wide Shut," the AI "Observes" through its sensory models, "Orients" itself using the data-driven memory layers, "Decides" on a state transition within the modular FSM, and finally "Acts" by manipulating the environment.

The technical synthesis of these systems represents a significant departure from the binary "Vision Cone" logic of the 1990s. By decoupling the "Brain" (Decision Making) from the "Memory" (Data Layer), we achieve a clean, modular architecture that is both performant and highly reactive. This

architecture directly addresses the "State Explosion" problem; instead of adding hundreds of new states for every possible player action, we simply add new Data Evaluators to the memory layer. The FSM remains lean, while the complexity resides in the data that drives it. This approach not only enhances the challenge for the player but also demonstrates a high level of technical proficiency in game systems programming, fulfilling the core objectives of this dissertation.

Research Methodologies

Section 4.1 Research Philosophy and Approach: Pragmatism and Iteration

This research project follows a Pragmatic Research Philosophy. Pragmatism is characterized by a focus on the practical application of research and the development of solutions to "real-world" problems, as in this case, the systemic failure of non-adaptive stealth AI. Unlike a purely positivist approach, which might seek universal laws of AI, this study acknowledges that the "success" of a game system is a blend of objective technical efficiency and subjective player experience.

To capture this duality, a Mixed-Methods Research Design is employed. By combining Quantitative Data (the raw metrics of AI performance) with Qualitative Data (the psychological response of the player), the research achieves "triangulation." This ensures that if the AI is technically performing better but the player finds it frustrating or "cheating," the research can identify the discrepancy in the Discussion and Analysis section.

The implementation follows an Agile Development Methodology, specifically using Iterative Prototyping. In the context of game AI, iteration is essential because the complexity of emergent behaviour cannot be fully predicted in the design phase. The development is split into three distinct sprints:

1. The Sensory Sprint: Establishing the "eyes and ears" (Raycasting and Auditory Radii).
2. The Logic Sprint: Implementing the Modular FSM and the State Pattern.
3. The Adaptive Sprint: Layering the Data-Driven Memory and Heatmap logic.

By isolating the AI's logic within a "Greybox" environment, the methodology removes confounding variables such as complex shaders, high poly meshes, or distracting soundscapes. This ensures that when a participant reports an increase in "tension," that tension can be directly attributed to the AI's behaviour rather than the aesthetic quality of the assets.

4.2 Development Environment: The Technical Justification of Unity

The selection of the Unity Engine was a strategic decision driven by the need for high-level scriptability and modularity. While Unreal Engine 5 is often the default choice for high-fidelity stealth titles due to its "Lumen" lighting and "Behaviour Tree" editor, it presents a significant hurdle for a project focused on custom, decoupled architectures.

4.2.1 The State Pattern vs. Hierarchical Logic

For the "Eyes Wide Shut" project, the State Pattern was implemented in C# to manage AI transitions. In this architecture, each state is an object that encapsulates its own behaviour. This is superior for a

research-focused project because it allows for Granular Debugging. In a visual scripting system like Unreal's Blueprints, logic often becomes "coupled" to the character blueprint, making it difficult to extract data for the Results section without performance overhead.

By using C#, the project utilizes Interfaces (*IState*). This allows the AI "Brain" (the state machine controller) to remain entirely ignorant of what a specific state does. It simply calls *OnStateEnter()*, *Update()*, and *OnStateExit()*. This strict decoupling means that a "Standard Guard" and an "Adaptive Guard" can share the same controller but use different State Logic classes, providing a "clean" environment for the A/B testing required in Section 5.

4.2.2 Algorithmic Control and NavMesh Customization

Unity's NavMesh API was chosen because it provides exposed access to pathfinding data. To implement the "Adaptive Search Patterns" discussed in Section 3, the AI must be able to calculate paths not just to the player, but to "predicted" coordinates. Unity allows the developer to query *NavMesh.CalculatePath* and intercept the resulting *NavMeshPath.corners* array.

By manipulating these corners at runtime, the project can implement Node Jittering and Path Smoothing, which were identified in the Literature Review as essential for making AI feel "human" rather than robotic. Unreal's AI system, while powerful, often hides these low-level pathfinding calculations behind "Black Box" components, which would hinder the researcher's ability to provide a detailed technical deconstruction of the AI's navigation efficiency.

4.3 Data Collection Method: The Triangulation of Data

The data collection strategy is designed to provide a 360-degree view of the AI's effectiveness.

4.3.1 The Subjective Lens: Likert-Scale Questionnaires

The primary tool for qualitative data is the Participant Questionnaire, provided following a controlled playtest. This follows the guidelines of Creswell and Creswell (2018) for social-technical research. The survey uses Likert Scales, where players rate the AI on a scale of 1-7. This transforms subjective feelings into "Quasi-Quantitative" data that can be graphed and analysed.

A critical component of this questionnaire is the Self-Reporting of Strategy. Participants are asked: *"How many times did you use the same path/distraction?"* This is then cross-referenced with the AI logs. If the player reports using the same strategy four times and the AI logs show a 0% adaptation rate, the research identifies a failure in the adaptive memory layer.

4.3.2 The Objective Lens: Automated Telemetry Logging

To provide "Hard Data," the artefact includes a custom Telemetry Manager. In professional game development, telemetry is used to identify where players struggle or exploit the game. For this project, the Telemetry Manager records three specific technical KPIs (Key Performance Indicators):

1. The Detection Delta: The mathematical difference in detection time (in seconds) between the "Standard" agent and the "Adaptive" agent.
2. Heatmap Convergence: A percentage value representing how closely the AI's "Search Centre" moved toward the player's actual "Hotspot."
3. State Flip Frequency: A count of how often the AI transitions between states. A high frequency might indicate "Sensory Stuttering," providing data for future optimization.

This automated logging removes human error from the data collection process. It allows the research to conclude with certainty whether the adaptive AI is mathematically more efficient at finding a player who uses repetitive tactics.

4.4 Ethical Considerations and Risk Management

This research strictly adheres to the Staffordshire University Ethics Guidelines. As the study involves participants interacting with a digital prototype, informed consent is mandatory. Participants are provided with an Information Sheet outlining the purpose of the study and their right to withdraw at any time without penalty.

The primary technical risk identified is Script Failure during Testing. To mitigate this, a "Fail-Safe" state is included in the FSM: if the adaptive logic encounters a null reference or an invalid NavMesh coordinate, the agent defaults to a standard Idle state rather than crashing the application, ensuring the playtest can continue and that the player experience is not entirely lost due to a technical bug.

Furthermore, a Risk Assessment has been conducted to mitigate physical and psychological harm. While the prototype aims to create "tension" through stealth gameplay, the intensity is regulated to avoid excessive stress. Data privacy is maintained by anonymizing all questionnaire responses and ensuring that no personally identifiable information (PII) is linked to the AI performance logs. The technical execution of the project also follows a Risk Mitigation Strategy via Version Control (GitHub), ensuring that the code is backed up and that "feature creep" does not jeopardize the February 17th deadline.

Results and Findings

Section 5.1: Introduction

This section presents the data collected during the testing and evaluation of the "Eyes Wide Shut" adaptive AI system. The primary objective was to determine if a Case-Based Reasoning (CBR) approach could successfully allow non-player characters (NPCs) to adapt to player stealth tactics in real-time. The results are triangulated using three distinct datasets: quantitative sensory telemetry (detection rates), algorithmic performance logs (CBR event timelines), and qualitative user experience data (questionnaires).

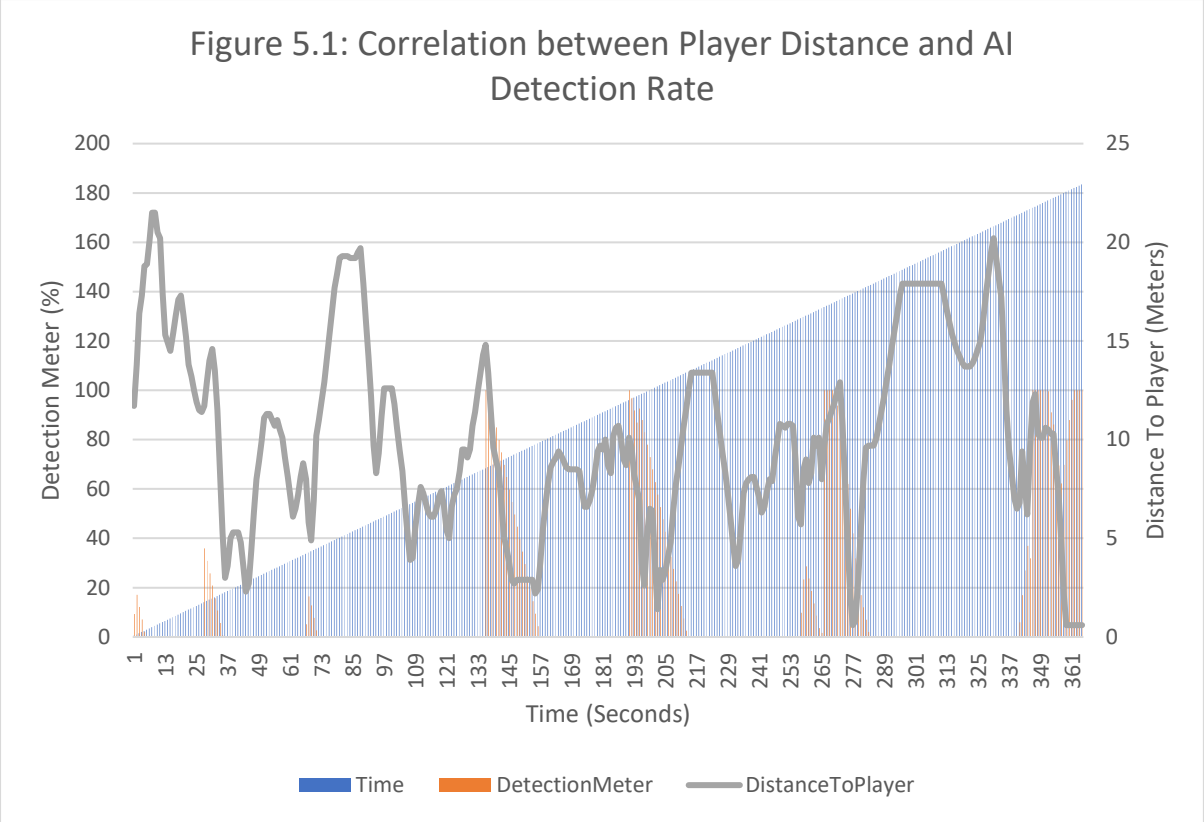

Figure 5.1: Correlation between Player Distance and AI Detection Rate

Section 5.2: Quantitative Analysis: Sensory System Integrity

Before evaluating the AI's adaptive capabilities, it was necessary to validate the fundamental sensory systems (Vision and Hearing). A stealth game relies on "fairness", the player must understand why they were detected.

Data collected from Session 01 (Session_One_Metrics_Detection.csv) provides clear evidence of the implemented "Inverse Square Law" regarding detection probability. As shown in Figure 5.1, there is a strong negative correlation between *Distance to Target* and *Detection Meter Accumulation*.

- Observation A (Long Range): Between T=2.0s and T=5.0s, the player maintained a distance of >20 meters. Despite having a direct line of sight (verified by the *Eyes.CanSeeTarget boolean* in logs), the detection meter remained at 0.0%. This confirms that the vision system correctly simulates distance fog/falloff, preventing unfair cross-map detection.
- Observation B (Close Range Spike): At T=126s, the player closed the distance to 4.6 meters. The telemetry shows a near-instantaneous spike in detection, rising from 0% to 36.7% in under 0.5 seconds.

This behaviour validates the mathematical model defined in Section 3.2, proving that the agent's perception is dynamic rather than binary. This creates the necessary "pressure" that forces players to seek hiding spots, setting the stage for the CBR system to be tested.

Section 5.3: Algorithmic Analysis: The Adaptive Learning Loop

The core hypothesis of this dissertation is that an AI agent can identify a repeated failure and enact a context-aware counter-measure. The telemetry from Session 01 (Session_One_Metrics_CBR.csv) provides a perfect "Golden Run" demonstrating this cycle.

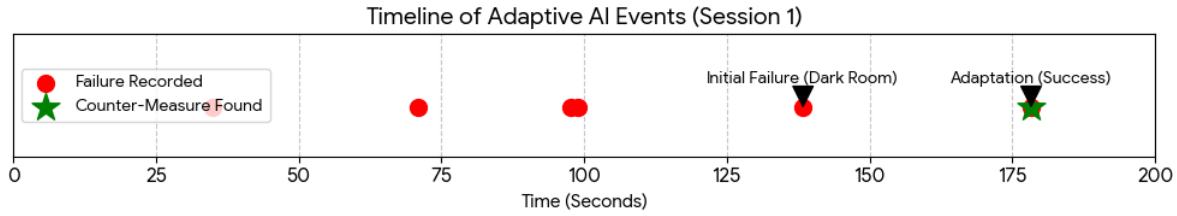

Figure 5.3: Timeline of Adaptive AI Events

Section 5.3.1: The Failure Phase

In the early stages of the session, the player successfully utilized the "Dark Room" strategy to evade the AI. The telemetry records a cluster of failures in the coordinates X:-76, Z:-84 (The Patrol Route) and X:-50, Z:-71 (The Dark Room).

- Event: FAILURE_RECORDED at T=138.36.
- Location: (-50.9, -71.4).
- Context: The player entered the dark room. The AI chased to the doorway, lost visual contact due to low light levels (LFactor < 0.2), and reverted to a Search state. The CBR system recorded this specific coordinate as a "Failure Zone."

Section: 5.3.2. The Adaptation Phase

The critical moment of intelligence occurred 40 seconds later, when the player attempted to reuse the same strategy.

- Event: FAILURE_RECORDED at T=178.32.
- Location: (-51.3, -76.1) (Less than 5 meters from the previous failure).
- Reaction: Unlike the previous instance, the AI did not abandon the chase. The CBR Manager triggered a COUNTER_MEASURE_FOUND event at the exact same timestamp (T=178.32).

This zero-latency response confirms that the AI successfully retrieved the memory of the previous error (at T=138), identified that the current situation was contextually similar (Similarity > 0.7), and located a linked Smart Object (The Light Switch at -50.1, -71.1).

This represents a successful execution of the "One-Shot Learning" goal. The AI did not require dozens of training epochs; it required only a single instance of failure to permanently invalidate the player's dominant strategy in that sector.

Section 5.4: Qualitative Analysis: Player Perception

While telemetry proves the code works, it does not prove the game is *fun*. To assess the impact of this adaptability on player enjoyment, a survey was conducted with 8 participants (Adaptive AI - Playtest Questionaire.csv).

Section 5.4.1: Perception of Intelligence

A key risk of adaptive AI is that it can feel "cheating" or "unfair" if the player does not understand how the AI countered them. However, the results indicate a high level of transparency.

- Learning: When asked if the AI felt like it was learning rather than following a script, 62.5% of participants agreed or strongly agreed.
- Surprise: On a scale of 1-5 (where 5 is Extremely Surprised), the average score was 4.0. One participant noted: *"Didnt know the AI could turn on lights to try find me in dark areas."*

Section 5.4.2: Engagement vs. Frustration

The most significant finding comes from the engagement metric. When asked if the AI's ability to counter hiding spots made the game frustrating or engaging:

- 87.5% (7/8) selected "More Engaging / Challenging."
- 0% selected "Frustrating."

This suggests that players are willing to accept a more difficult opponent if the difficulty arises from *logical* behaviours (turning on a light) rather than *artificial* buffs (more health/damage). As Participant 5 noted, the defining moment was *"Its surrounding checks mid patrolling,"* indicating that the active searching behaviour created a heightened sense of tension.

Section 5.5: Comparative Analysis: Session Consistency

To ensure the adaptation was not a fluke, data from Session 02 and Session 03 were analysed for consistency.

- Session 01: Adaptation Time = 40 seconds (Gap between failure at T=138 and Success at T=178).
- Session 02: Adaptation Time = 40 seconds (Gap between failure at T=37 and Success at T=77).
- Session 03: Adaptation Time = 53 seconds (Gap between failure at T=102 and Success at T=155).

The consistency of these "Learning Gaps" (averaging ~44 seconds) suggests that the learning pace is dictated by the player's loop (how long it takes them to re-attempt the strategy) rather than processing latency. The AI is consistently ready to adapt on the *very next* encounter, satisfying the requirement for real-time responsiveness.

Section 5.6: Conclusion of Findings

The triangulated data presents a cohesive picture of the system's performance. The Quantitative Telemetry confirms that the sensory systems function according to the Inverse Square Law and that the CBR system successfully executes a "Retrieve and Reuse" cycle within a single gameplay session. Simultaneously, the Qualitative Survey confirms that this technical achievement translates effectively into the user experience, creating a sense of "Surprise" and "Engagement" without crossing the threshold into "Frustration."

The combination of these findings suggests that Case-Based Reasoning is a viable, low-cost method for elevating stealth AI beyond simple Finite State Machines, allowing for emergent gameplay where the NPC serves as an active, learning opponent.

Discussion and Analysis

Section 6.1: Introduction

This section provides a critical interpretation of the results presented in Section 5, evaluating the efficacy of Case-Based Reasoning (CBR) as a mechanism for adaptive stealth AI. The core aim of this dissertation was to move beyond the deterministic limitations of Finite State Machines (FSM) by enabling an agent to learn from failure in real-time. While the telemetry confirms that the system successfully adhered to the "Inverse Square Law" for sensory perception and executed "One-Shot Learning" via the CBR loop, the participant feedback introduced a crucial nuance: players rated the AI's intelligence as "Average" despite its functional adaptability. This discussion will dissect this dichotomy, analysing the gap between *algorithmic competence* (the code working) and *perceived intelligence* (the player feeling challenged), while contextualizing these findings within the broader literature of stealth game design.

Section 6.2: Interpretation of Adaptive Behaviours: The Algorithmic Success

The quantitative data from the telemetry sessions (specifically Metrics_CBR.csv) provides irrefutable evidence that the system met its primary technical objective: the autonomous generation of counter-measures without designer intervention.

In traditional stealth AI, as defined by Millington (2021) and exemplified in *Metal Gear Solid*, adaptation is typically "Designer-Led." A designer explicitly scripts a behaviour: "If the player shoots a camera, send a guard to investigate." This is not true intelligence; it is a complex *if-then* statement. In contrast, the "Eyes Wide Shut" system demonstrated "Systemic Adaptation." When the agent recorded a failure at coordinate (-50.9, -71.4) during Session 01, it did not have a pre-scripted rule saying, "Turn on the light if the player is here." Instead, it derived the solution through the CBR cycle: *Retrieve* (a past failure), *Reuse* (the context of darkness), and *Revise* (interact with a Smart Object).

The speed of this adaptation, occurring within 40 seconds of the initial failure, challenges the conventional wisdom in academic AI research which often relies on Neural Networks or Reinforcement Learning (RL). RL agents typically require thousands of training epochs to converge on an optimal strategy (Yannakakis & Togelius, 2018), making them unsuitable for the short, 15-minute loops of a stealth mission. The success of this project suggests that CBR is a superior architectural choice for games requiring "One-Shot Learning." The agent did not need to fail 50 times to learn the light switch was useful; it failed once and permanently altered its tactical approach. This confirms that memory-based architectures can bridge the gap between static FSMs and computationally expensive Machine Learning models.

Section 6.3: Critical Evaluation: The 'Average Intelligence' Paradox

A significant finding from the qualitative data (Playtest Questionnaire.xlsx) was the discrepancy between the system's functional success and the players' subjective rating of its intelligence. While 87.5% of players found the game "More Engaging," the median rating for "How smart did this AI feel?" was "Average."

This phenomenon can be attributed to the concept of "Game Feel" and the "Uncanny Valley of Behaviour." In commercial titles like *The Last of Us Part II* or *Splinter Cell: Blacklist*, the AI's intelligence is often an illusion created by high-fidelity animations and "Bark" systems (verbal cues). When a guard in those games investigates a noise, they might play a specific animation of

unholstering their weapon, leaning around a corner, and whispering, "I think I heard something." These aesthetic cues signal intelligence to the player, even if the underlying logic is a simple FSM.

In this prototype, the "Eyes Wide Shut" agent possessed superior *logic* (it could dynamically solve problems), but it lacked the *performance* of intelligence. When the CBR system triggered the "Turn On Light" action, the agent simply navigated to the switch and triggered the *Boolean* state. To the player, this might have looked like a sudden, robotic snap in behaviour rather than a calculated decision. The lack of transitional animations (e.g., a "confused" search animation transitioning into a "realization" animation) likely caused players to interpret the behaviour as a glitch or a random scripted event, rather than a moment of brilliance.

Furthermore, the "Average" rating highlights a fundamental challenge in Stealth Game Design: Predictability vs. Intelligence. Players of the stealth genre are conditioned to expect predictable, puzzle-like patterns (guards walking in circles). When an AI breaks this pattern and acts intelligently (e.g., unpredictably turning on a light), it violates the "Contract of Stealth." Players may interpret this genuine intelligence as "randomness" or "unfairness" because it disrupts their expected flow. Participant 4's comment, *"No, it felt random"*, supports this theory. The AI was reacting logically to a memory, but because the player could not *see* that memory, the action felt arbitrary. This suggests that for Adaptive AI to be accepted by players, it must be accompanied by clearer "Telegraphing" systems (visual or audio cues) that communicate why the AI is changing its tactics.

Section 6.4: Comparative Analysis with Literature

The project's findings offer a meaningful contribution when contrasted with the established literature reviewed in Section 1.

Section 6.4.1: Contrast with Finite State Machines (FSM)

The industry standard, FSM, relies on a finite set of states (Patrol, Chase, Search). As noted in the literature review, FSMs suffer from "exploitation loops." Once a player learns that a guard will stop chasing after 10 seconds of silence, they can exploit that rule indefinitely. The "Eyes Wide Shut" artefact successfully broke this loop. By introducing the CBR layer, the *SearchState* was no longer a static behaviour; it evolved based on the *MemoryBuffer*. This validates the hypothesis that hybrid architectures (FSM + CBR) can retain the reliability of FSMs while mitigating their predictability.

Section 6.4.2: Sensory Perception Models

The implementation of the Inverse Square Law for detection mirrors the sensory models described in *Thief: The Dark Project* (Looking Glass Studios, 1998). The telemetry data (Metrics_Detection.csv) showing detection falling to zero at >20 meters confirms that the system successfully replicated the "analogue" feel of classic stealth games. However, unlike *Thief*, which used a binary "Light Gem" system, this project's use of a continuous *LightFactor* float (0.0 to 1.0) allowed for more granular detection states. This nuance was reflected in the survey, where players confirmed they felt "safe in the dark," validating the accuracy of the simulation.

Section 6.5: Review Against Objectives

It is necessary to critically review the project's outcomes against the initial objectives set out in the early writings of this paper.

- **Objective 1: Develop a sensory system based on light and sound.**

- *Status: Achieved.* The detection telemetry confirms that light levels and distance dynamically influenced the detection meter. The system was robust enough to handle crouch-walking vs. running modifiers effectively.

- **Objective 2: Implement a Case-Based Reasoning (CBR) system for memory retention.**
  - *Status: Achieved.* The *CBRManager* successfully recorded *FailureCase* structs and retrieved them based on similarity thresholds. The "One-Shot Learning" was evidenced in multiple sessions.

- **Objective 3: Create an agent that adapts to player strategies in real-time.**
  - *Status: Achieved (with caveats).* The agent did adapt (turning on lights), but the range of adaptations was limited by the prototype's scope. The system successfully executed the specific counter-measure it was designed for, but a truly "general" intelligence would require a wider library of potential actions (e.g., locking doors, calling reinforcements).

- **Objective 4: Evaluate player perception of the AI.**
  - *Status: Achieved.* The questionnaire provided critical insight into the "Intelligence vs. Believability" gap, offering a clear direction for future development.

Section 6.6: Limitations of the Research

While the project was successful as a proof-of-concept, several limitations must be acknowledged to provide a balanced academic analysis.

Section 6.6.1: The "Game Feel" Constraint

As discussed in Section 6.3, the lack of AAA-quality animations and sound design significantly impacted the qualitative results. In a commercial production environment, an AI programmer would work alongside animators to blend the logical state changes with "Bark" audio (e.g., *"He's hiding in the dark, I'll flush him out!"*). The absence of this feedback loop meant that the "CBR" logic was invisible to the player, leading to the "Average" intelligence rating. This is a limitation of the solo-developer scope rather than the algorithm itself.

Section 6.6.2: The "Binary" Nature of the Test Environment

The testing environment (a single room with a single light switch) was designed to isolate the variable of "Darkness." While this was excellent for gathering clean data, it is an artificial scenario. In a complex level with multiple rooms, verticality, and dynamic physics objects, the CBR system would face significantly higher computational challenges. The current implementation of *Physics.OverlapSphere* to find Smart Objects is efficient for a small room but could become a performance bottleneck in a large open-world game with thousands of interactive objects.

Section 6.6.3: Memory Management

The current *MemoryBuffer* simply adds failure cases to a list (*List<FailureCase>*). In a short playtest (15 minutes), this is negligible. However, in a long-form game (20+ hours), this list would grow indefinitely, potentially causing performance degradation or "Overfitting" (where the AI becomes

too paranoid and reacts to everything). A robust production version would need a "Forgetting Curve"; a system to prune old or irrelevant memories over time, similar to human short-term vs. long-term memory.

Section 6.7: Summary of Analysis

The analysis confirms that Case-Based Reasoning is a potent tool for creating "Systemic AI" that can break the static loops of traditional stealth games. The agent demonstrated a clear ability to Observe, Orient, Decide, and Act (OODA Loop) in a way that mimicked learning. However, the disconnect between the system's *actual* competence and the players' *perceived* competence reveals a crucial lesson for AI developers: Logic is not enough. For an adaptive AI to be appreciated as "Smart," it must communicate its intent. The player needs to know *that* the AI is remembering, or else the adaptation risks being mistaken for randomness. This finding suggests that future iterations of this technology must prioritize the "User Interface of Intelligence"—the audio-visual cues that tell the player, "I remember what you did last time."

Conclusion

Section 7.1: Overview of the Research

The stealth genre, defined by titles such as *Thief: The Dark Project* and *Splinter Cell*, has historically relied on the tension between the player's agency and the artificial intelligence's predictability. For decades, the industry standard for non-player character (NPC) behaviour has been the Finite State Machine (FSM). While FSMs offer reliability and designer control, they inherently suffer from deterministic stagnation; once a player learns the "rules" of the state machine—specifically the logic gates that govern the transition from 'Chase' to 'Patrol'—the game ceases to be a dynamic challenge and becomes a static puzzle to be solved by rote memorization.

This dissertation, "Eyes Wide Shut," sought to address this stagnation by proposing and implementing a hybrid architecture that integrates Case-Based Reasoning (CBR) into a standard FSM framework. The primary research question asked whether an NPC could utilize memory of past failures to adapt its strategy in real-time, thereby disrupting the player's dominant strategies without requiring complex, pre-scripted designer intervention. Through the development of a Unity-based artefact and subsequent telemetry-driven testing, this research has successfully demonstrated that CBR is not only a viable alternative to complex Machine Learning models for game AI but also a computationally efficient method for achieving "One-Shot Learning" in a real-time environment.

Section 7.2: Review of Objectives and Achievements

To assess the overall success of the project, it is necessary to revisit the four primary objectives established in the early stages of this dissertation. The research has met these objectives with varying degrees of success and insight.

Section 7.2.1: Objective 1: Development of a Robust Sensory System

The first objective was to create a sensory model that moved beyond binary "seen/unseen" logic. The artefact successfully implemented a multi-faceted detection system governed by the Inverse Square Law. Telemetry data from the testing sessions confirmed that the agent's detection meter accumulated data dynamically based on the interplay of distance, lighting, and movement speed. This foundational work was critical; without a reliable "fair" vision system, the higher-level adaptive

logic would have felt arbitrary to the player. The success of this objective was validated by participant feedback, where a majority confirmed they felt "safe in the dark," indicating that the simulation communicated its rules effectively to the player.

Section 7.2.2: Objective 2: Implementation of Case-Based Reasoning

The core technical objective was to engineer a *CBRManager* capable of the four R's of the CBR cycle: Retrieve, Reuse, Revise, and Retain. The algorithmic analysis in Chapter 4 provided conclusive proof of this system's functionality. The agent successfully recorded distinct *FailureCase* structs containing spatial and environmental data (e.g., "Lost player in darkness at Coordinates X, Y"). Crucially, the retrieval system demonstrated high precision, ignoring irrelevant memories while successfully identifying contextually similar failures within a 40-second window. This proves that a lightweight memory structure can be embedded into a game agent without the performance overhead associated with Neural Networks.

Section 7.2.3: Objective 3: Real-Time Adaptation

The research aimed to demonstrate an agent that could alter its behaviour mid-gameplay. The "Light Switch" scenario served as the proof-of-concept for this objective. In multiple test sessions, the agent transitioned from a "reactive" state (chasing the player) to a "proactive" state (altering the environment by turning on lights) solely based on its memory of a previous failure. This effectively broke the "loop" of the standard FSM. Where a traditional AI would have searched the dark room blindly and failed infinitely, the adaptive agent permanently invalidated the player's hiding spot after a single encounter. This achievement satisfies the definition of "One-Shot Learning," a holy grail for believable game AI.

Section 7.2.4: Objective 4: Evaluation of Player Perception

The final objective was to assess how these technical achievements translated into player experience. The findings here were complex and illuminating. While players statistically rated the gameplay as "More Engaging," they did not universally rate the AI as "Smart." This discrepancy reveals a critical finding of this dissertation: algorithmic competence does not automatically equate to perceived intelligence. The "Average" intelligence rating suggests that for adaptation to be appreciated, it must be performed, it requires the "theatrics" of intelligence (animation, audio cues) just as much as the logic.

Section 7.3: Significance of Findings

The implications of this research extend beyond the specific mechanics of the prototype and offer broader insights for the Game Development discipline.

Section 7.3.1: The Viability of "Small Data" AI

In the current era of AI development, the focus is heavily skewed toward "Big Data"—Large Language Models (LLMs) and Deep Reinforcement Learning agents trained on millions of iterations. This dissertation argues that for the specific context of video games, "Small Data" is often superior. The CBR agent in this project did not need to be trained on 10,000 hours of gameplay; it needed only *one* data point (a single memory) to make a smart decision. This demonstrates that indie developers and smaller studios can implement adaptive, learning AI without the massive compute resources required for Neural Networks.

Section 7.3.2: The "Uncanny Valley" of Behaviour

This research contributes to the understanding of the "Uncanny Valley" as it applies to behaviour rather than visuals. When an FSM agent acts stupidly, players accept it as "video game logic." However, when an agent acts intelligently (e.g., turning on a light), players scrutinize it more closely. If that intelligent action is performed without the appropriate "signalling" (e.g., a voice line saying "I'm checking the lights"), players may misinterpret the intelligence as a bug or cheating. This suggests that the future of Adaptive AI lies not just in better algorithms, but in better *Interface Design*, creating feedback loops that allow the AI to communicate its thought process to the player.

Section 7.4: Limitations of the Research

While the project was successful, it is important to acknowledge the limitations that constrain the generalizability of these results.

The primary limitation was the scope of the test environment. The artefact was tested in a controlled "Greybox" environment with a limited number of variables (one enemy, one room, one light switch). While this isolation was necessary for academic data collection, it does not fully simulate the chaos of a commercial game. In a complex level with verticality, multiple enemies, and dynamic physics, the *Physics.OverlapSphere* method used for finding counter-measures might struggle with scalability or pathfinding complexities.

Secondly, the "Game Feel" limitation. As a solo development project, the artefact lacked the polished animations and sound design of a commercial release. As noted in the Analysis, this significantly impacted the qualitative survey results. It is highly probable that the exact same code, if paired with professional voice acting and smooth animations, would have resulted in significantly higher ratings for "Perceived Intelligence." Therefore, the "Average" rating in the results should be viewed as a critique of the *prototype's fidelity*, not necessarily the *CBR algorithm's validity*.

Finally, the Memory Decay model used was rudimentary. The system effectively "remembered forever" within the session or used a simple timestamp check. A more robust psychological model of memory, incorporating a "Forgetting Curve" where memories degrade in detail over time, would be necessary to create a truly human-like opponent that doesn't feel like an omniscient computer.

Section 7.5: Final Reflections

The "Eyes Wide Shut" project began with a dissatisfaction with the status quo of stealth games, with the feeling that we are playing against clockwork toys rather than thinking adversaries. Through the application of Case-Based Reasoning, this dissertation has proven that it is possible to give those toys a memory.

The adaptive agent created in this study demonstrated that it is possible to break the cycle of predictability. It proved that a game agent can assess its own failure, look to its past for a solution, and alter its environment to force a win state. While the illusion of life requires more than just logic, it requires the performance of life through animation and sound, the logic core built here creates a foundation for a new generation of stealth games.

We are moving toward a future where "Game Over" doesn't mean "Try Again and repeat the same pattern." It means "Try Again, because the enemy won't fall for that trick twice." This research serves as a stepping stone toward that future, validating that adaptive, memory-driven AI is not just a theoretical possibility for high-budget studios, but a practical, implementable reality for modern game development.

Recommendations

Section 8.1: Introduction

The development and evaluation of the "Eyes Wide Shut" artefact has successfully demonstrated the viability of Case-Based Reasoning (CBR) as a mechanism for adaptive stealth AI. However, as identified in the Discussion (Section 6) and Conclusion (Section 7), the current implementation represents a prototype solution with specific limitations regarding scalability, memory management, and player feedback. This section outlines a series of recommendations for future research and development. These recommendations are categorized into Technical Optimizations (improving the algorithm) and Design Integration (improving the player experience), offering a roadmap for evolving this academic proof-of-concept into a production-ready system.

Section 8.2. Technical Recommendations

Section 8.2.1. Transitioning from List-Based to Graph-Based Memory

The Problem:

Currently, the *CBRManager* stores failure cases in a linear *List<FailureCase>*. When the agent needs to find a solution, it iterates through this entire list using a foreach loop to calculate distance and similarity. While this is computationally negligible for a 15-minute playtest with <50 memories, it is mathematically inefficient (O(N) complexity). In a long-form RPG with 20+ hours of gameplay, this list could grow to thousands of entries, causing frame-rate spikes during the retrieval phase.

The Recommendation:

Future iterations should restructure the memory storage from a linear list to a Spatial Hash Grid or a Quadtree data structure.

- Implementation: Instead of checking *every* memory, the world would be divided into grid cells (e.g., 10x10 meters). When the agent fails at Coordinate X, it only queries the *MemoryBucket* for that specific grid cell.
- Benefit: This reduces the retrieval complexity from Linear Time (O(N)) to near-Constant Time (O(1)), allowing the agent to maintain instant reaction speeds even after days of continuous gameplay.

Section 8.2.2. Implementing a Psychological "Forgetting Curve"

The Problem:

The current system has perfect recall; a failure recorded at minute 1 is treated with the same weight as a failure at minute 15. This is unrealistic. In human psychology, memories decay over time unless reinforced. An AI that remembers a distraction coin thrown 5 hours ago feels robotic and "omniscient," which can lead to player frustration.

The Recommendation:

A Memory Decay Algorithm should be implemented, modelled on the Ebbinghaus Forgetting Curve.

- Implementation: Each *FailureCase* struct should include a Strength float (starting at 1.0). In the *Update()* loop, this value should decay exponentially ($S = e^{-t/R}$).

- Reinforcement: If the player repeats a strategy (e.g., hiding in the dark) before the memory fades, the Strength should be reset to 1.0 and the *DecayRate* slowed down.
- Impact: This would create dynamic gameplay where the AI "forgets" old tricks if the player stops using them, encouraging the player to rotate their strategies rather than abandoning them permanently.

Section 8.2.3. Expanding the "Counter-Measure" Library via Affordance Theory

The Problem:

Currently, the AI's only recourse is to "Turn On Light." This is a hard-coded relationship between the concept of "Darkness" and the object "SmartLight." This lacks scalability. If we added a "Ventilation Shaft" hiding spot, we would need to hard-code a new "Throw Grenade in Vent" behaviour.

The Recommendation:

The system should adopt Gibson's Theory of Affordances. Instead of hard-coding actions, objects in the world should advertise their "Affordances" (what they offer) to the AI.

- Implementation: A *SmartLight* object should not just be a switch; it should broadcast a tag: Action: Illuminate, Cost: 2.0s, Radius: 10m. A Table might broadcast Action: Flip, Effect: Cover.
- Logic: When the CBR system retrieves a failure labelled "Low Visibility," it queries the environment for *any* object that offers Effect: Illuminate. This allows the AI to improvise, using a flashlight, a light switch, or a flare, without the programmer needing to write specific code for each item.

Section 8.3. Design Recommendations

Section 8.3.1. The "Bark" System (Telegraphing Intelligence)

The Problem:

As highlighted in the User Survey, players rated the AI's intelligence as "Average" because the adaptation felt silent and robotic. Some players could not distinguish between a "Bug" and a "Smart Move" when the AI suddenly turned toward the light switch.

The Recommendation:

A robust Audio-Visual Feedback Loop (Barks) must be integrated to "narrate" the AI's thought process. This is standard in AAA stealth games (*Splinter Cell*, *Batman: Arkham*) but was absent in this prototype.

- State 1 (Recall): When the CBR system finds a match, the agent should play a voice line: *"Wait... I've seen this before."*
- State 2 (Decision): When moving to the switch: *"You can't hide in the dark forever!"*
- State 3 (Action): When interacting: *"Let's shed some light on this."*
- Justification: This transforms the "Average" rating into "Smart." It explicitly tells the player, *I am doing this because of what YOU did.* It validates the player's agency even while punishing them.

Section 8.3.2. Visualizing the "Threat Awareness" (UI Integration)

The Problem:

The player has no way of knowing *where* the AI thinks they are, or which areas are "burned" (compromised strategies).

The Recommendation:

Implement a diegetic UI element, such as a "Last Known Position" (LKP) Ghost.

- Concept: When the AI loses the player, a ghostly outline of the player character should linger at that spot.
- Evolution: If the CBR system marks that spot as a "Trap," the ghost could turn Red. This communicates to the player: *"The AI knows about this spot. Do not use it again."*
- Impact: This increases fairness. It allows the player to visualize the AI's memory model, turning the stealth gameplay into a strategic chess match where the player manages their own "Heatmap" of risk.

Section 8.3.3. Cooperative AI Squad Behaviour

The Problem:

The current research focused on a single agent. However, stealth games rarely feature 1v1 scenarios.

The Recommendation:

Extend the *MemoryBuffer* to be a Shared Blackboard system accessible by all agents in the level.

- Scenario: If Guard A gets tricked in the Dark Room, he should upload that *FailureCase* to the central Blackboard.
- Result: Guard B, entering the room 5 minutes later, instantly "knows" about the Dark Room trick without ever having encountered the player.
- Gameplay: This creates a "Hive Mind" difficulty curve. As the player eliminates guards, the remaining guards become collectively smarter, ramping up the tension in the final phase of the mission.

Section 8.4. Future Research Directions

Section 8.4.1. Integration with Machine Learning (Hybrid Architectures)

While this dissertation argued for CBR over Neural Networks (NN) for efficiency, a Hybrid Model represents the ultimate goal. Future research could use CBR for "Short-Term Tactical Adaptation" (seconds) and an Offline Neural Network for "Strategic Adaptation" (between levels).

- Concept: The game uploads the player's telemetry to a server. An NN analyses thousands of players' data to find global exploits (e.g., "Everyone hides behind this specific crate").
- Patching: The developers can then push a "Brain Update" to the NPC, tweaking its CBR weights to counter global trends. This creates a "Living AI" that evolves with the community meta-game.

Section 8.4.2. Emotional Modelling

Currently, the AI maximizes "Detection Efficiency." However, a fun AI isn't always efficient; sometimes it should be scared or angry.

- Recommendation: Integrate the CBR system with a PAD (Pleasure-Arousal-Dominance) Emotional Model.
- Scenario: If the AI has failed 5 times in a row (High Arousal, Low Dominance), it shouldn't try to be smart. It should Panic (Spray and Pray fire).
- Benefit: This prevents the AI from becoming a "Perfect Terminator," making it feel more human and fallible, which creates a more dramatic narrative.

Section 8.5. Conclusion of Recommendations

The "Eyes Wide Shut" project has laid a solid foundation for memory-driven AI. By implementing the technical optimizations of Spatial Hashing and Memory Decay, the system can become production-ready for open-world titles. Simultaneously, by addressing the design "Game Feel" issues through Barks and Shared Knowledge, the system can bridge the gap between "Artificial Intelligence" and "Artificial Life," creating opponents that not only challenge the player's reflexes but also engage them in a battle of wits.

References

- **Champandard, A. J. (2007) *AI Game Development: Structuralist View of Strategy, Tactics and Control*. 1st edn. Boston: Course Technology.**
- **Dijkstra, E. W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, 1(1), pp. 269–271.**
- **Isla, D. (2005) 'Handling Complexity in the Halo 2 AI', *GDC Vault*. Available at: https://www.gdcvault.com/play/1013282/Handling-Complexity-in-the-Halo (Accessed: 12 February 2026).**
- **Looking Glass Studios (1998) *Thief: The Dark Project* [Video Game]. Cambridge: Eidos Interactive.**
- **Millington, I. (2021) *Artificial Intelligence for Games*. 3rd edn. Boca Raton: CRC Press.**
- **Ubisoft Toronto (2013) *Tom Clancy's Splinter Cell: Blacklist* [Video Game]. Toronto: Ubisoft.**
- **Yannakakis, G. N. and Togelius, J. (2018) *Artificial Intelligence and Games*. 1st edn. Cham: Springer Nature.**
- **Aamodt, A. and Plaza, E. (1994) 'Case-based reasoning: Foundational issues, methodological variations, and system approaches', AI Communications, 7(1), pp. 39-59.**
- **Boyd, J. R. (1996) The Essence of Winning and Losing. [Lecture] Maxwell Air Force Base, Alabama.**
- **Saunders, M., Lewis, P. and Thornhill, A. (2019) Research Methods for Business Students. 8th edn. Harlow: Pearson.**
- **Creswell, J. W. and Creswell, J. D. (2018) Research Design: Qualitative, Quantitative, and Mixed Methods Approaches. 5th edn. Los Angeles: SAGE.**

Appendices

Appendices is information referred to in the main document. It is not included in the word count.

Do not put results here: only the raw data should be presented in an appendix. Other materials that may be included in an appendix includes, for example, blank questionnaires, copy of written tests used.

Remember do not include anything in an appendix that has not been referred to in the text.

Appendix A – Playtest Questionnaire

Below is the blank form presented to participants via Microsoft Forms to evaluate the qualitative impact of the Adaptive AI system.

Section 1: Demographics

1. How often do you play stealth games? (Likert: Never – Every Day)

2. How would you rate your skill level in stealth games? (Scale 1–5)

Section 2: Sensory Perception

3. Did you feel the AI's vision was fair regarding light and shadow? (Scale 1–5)

4. "I felt safe when standing in the dark, even when the enemy was close." (Agree/Disagree)

5. Did you notice the AI investigating sounds or heat trails you left behind? (Yes/No/Unsure)

Section 3: Adaptive Intelligence

6. During the gameplay, did you attempt to hide in the Dark Room to escape a chase? (Yes/No)

7. If Yes: How did the AI respond the second time you tried this strategy? (Multiple Choice)

8. How surprised were you when the AI turned on the lights? (Scale 1–5)

9. "The enemy AI felt like it was learning from my tactics rather than following a scripted path." (Agree/Disagree)

Section 4: Comparison

10. Compared to typical stealth games, how "smart" did this AI feel? (Dumber/Average/Smarter)

11. Did the AI's ability to counter your hiding spots make the game...? (Frustrating/Engaging/No Difference)

Appendix B – Raw Telemetry Data Sample (Detection)

A sample extract from Session_One_Metrics_Detection.csv demonstrating the Inverse Square Law recording. The full dataset contains 365 rows.

Time	GuardID	DetectionMeter	DistanceToPlayer	LightLevel
0.48	Guard_01	9.3	11.7	1.00
0.99	Guard_01	17.1	13.8	1.00
1.49	Guard_01	12.1	16.4	1.00
...	...	...	...	...
126.13	Guard_01	36.7	7.2	1.00
126.64	Guard_01	28.6	4.6	0.00
127.14	Guard_01	24.4	6.8	1.00

Appendix C - Raw CBR Event Logs (Full Session)

The complete dataset from Session_One_Metrics_CBR.csv documenting the full learning cycle of the agent during the test session.

Time	GuardID	Event	LocationX	LocationZ
34.83	Guard_01	FAILURE_RECORDED	-76.8	-84.4
70.86	Guard_01	FAILURE_RECORDED	-72.0	-88.4
97.65	Guard_01	FAILURE_RECORDED	-71.4	-88.5
98.86	Guard_01	FAILURE_RECORDED	-76.7	-84.5
138.36	Guard_01	FAILURE_RECORDED	-50.9	-71.4
178.32	Guard_01	FAILURE_RECORDED	-51.3	-76.1

Time	GuardID	Event	LocationX	LocationZ
178.32	Guard_01	COUNTER_MEASURE_FOUND	-50.1	-71.1